Oppgave 1 – Modellering (25%, ~1 time)

Problembeskrivelse

En restaurant har behov for en database for å lagre informasjon om matrettene som blir tilbudt (dvs., menyen). Hver matrett har navn og pris.

Hver matrett skal også tilhøre en (og bare en) "kategori" som f.eks. "forrett", "hovedrett" eller "dessert". Disse kategoriene skal lagres i en egen tabell for "lovlige" kategorier.

Hver matrett kan videre inneholde et vilkårlig antall allergener. Disse allergenene skal lagres i en egen tabell for "lovlige" allergener.

(Denne problembeskrivelsen gjelder bare for denne oppgaven med mindre noe annet er nevnt.)

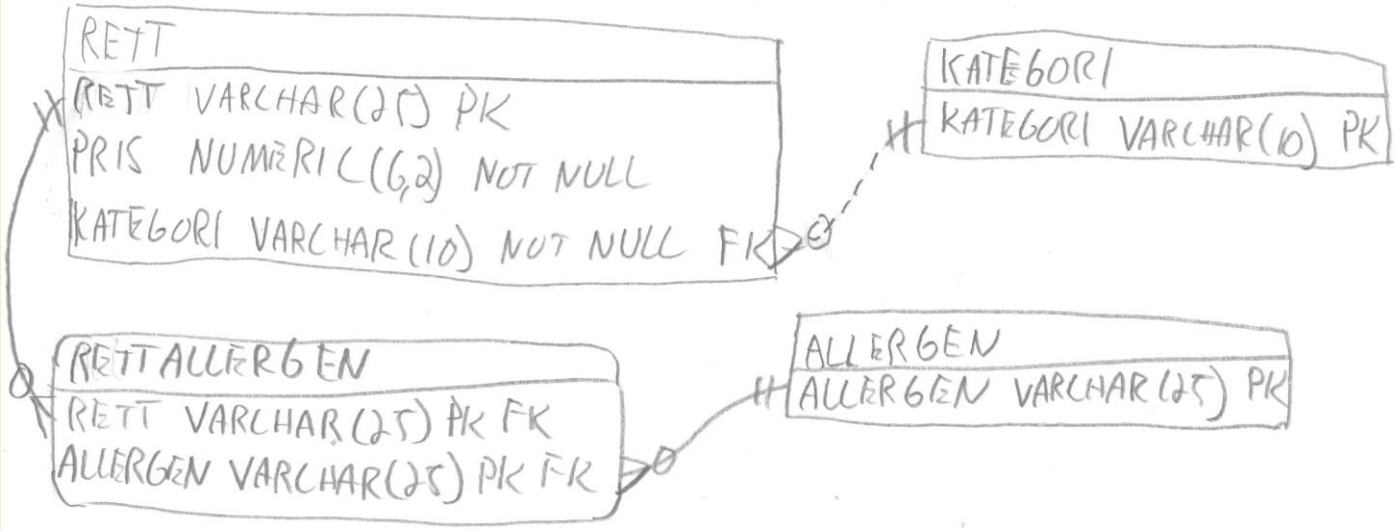
Oppgave

Lag (tegn) en logisk (fullstendig) ER-modell for denne databasen i en av notasjonene som har vært brukt i kurset.

Huskeliste (for hva besvarelsen minst skal inneholde)

- Bestem hva som er hensiktsmessige tabeller for denne databasen.
- Primærnøkler (som du mener er mest hensiktsmessig)
- Fremmednøkler
- Datatyper
- Min/maks kardinalitet
- Sterke/svake entitetstyper og eksistensavhengighet/uavhengighet (kråkefot-notasjon) eller eventuell type aggregering (UML-notasjon)
- Rettferdiggjøre at løsningen tilfredsstiller hver av 1., 2., og 3. normalform

Løsningsforslag



Det må også være en fornuftig rettferdiggjøring for hvorfor databasen er minst på 3. normalform.

Oppgave 2 – SQL (25%, ~1 time)

Problembeskrivelse

Oppgaven baserer seg løst på problembeskrivelsen til oppgave 1. Anta at en forenklet/ redusert/ minimal løsning kan bestå av disse to tabellene:

Rett(navn, pris, kategori*)
Kategori(kategori)

Oppgave

a) Skriv SQL kode for å opprette tabellen Rett. Du kan anta at tabellen Kategori allerede eksisterer. Husk datatyper, primærnøkler, not/not null og eventuelle fremmednøkler med on update/delete
b) Skriv SQL kode for å legge til en ny kolonne Antall_bestilt (som teller hvor mange ganger retten har blitt bestilt) til tabellen Rett
c) Skriv SQL kode for å fjerne kolonnen Antall_bestilt fra tabellen Rett
d) Skriv SQL kode for å legge inn en ny rett. Du kan selv velge data
e) Skriv SQL kode for å fjerne/slette den retten du nettopp la inn i forrige delspørsmål
f) Skriv SQL kode for å finne alle retter av kategorien "Hovedrett"
g) Skriv også forrige delspørsmål som relasjonsalgebra. Potensielt nyttige operatorer til å bruke i svar: π, σ, $\cup$, $\cap$, $\times$, $\bowtie$, $\ltimes$, $\rtimes$ og $\mapsto$
h) Skriv SQL kode for å finne antall retter av hver kategori
i) Skriv SQL kode for å finne eventuelle kategorier som ikke har noen retter
j) Forklar de "praktiske" forskjellene mellom inner, left outer, right outer og full outer join

Løsningsforslag

a) Create table rett(
navn varchar(25) primary key,
pris numeric(6,2) not null,
kategori varchar(10) not null
references kategori(kategori) on delete restrict on update cascade
);

b) Alter table rett add column antall_bestilt int;

c) Alter table rett drop column antall_bestilt;

d) Insert into rett values ('Pizza', 49.99, 'Hovedrett';

e) Delete from rett where rett='Pizza';

f) Select * from rett where kategori='Hovedrett';

g) $\sigma_{\text{kategori='Hoverett'}}$(rett)

h) Select kategori, count(*) from rett group by kategori;

i) Select * from kategori where kategori not in (select kategori from rett);

j) Inner join krever at det skal være en "match" på verdiene det joines på. Dvs., en rett uten kategori (nullmerke) vil ikke bli tatt med i en spørring med en inner join mellom tabellene Rett og Kategori. En left eller right outer join vil derimot ta med den "umatchede" verdien som er i henholdsvis venstre eller høyre tabell. Dermed kan man få med retter uten kategorier. En full outer join vil ta med umatchede verdier i både venstre og høyre tabell. Dvs., man vil få med både retter uten matchende kategori, og kategorier uten retter som har denne kategorien. Det er veldig sjelden man i praksis har bruk for en full outer join.

Oppgave 3 – ORM/JPA (25%, ~1 time)

Vi har ansvar for en liten kafé, og ønsker å holde oversikt over **matrett**er og **ingrediens**er, f.eks. med tanke på å kunne liste allergener (= ingredienser som er allergifremkallende) for ulike matretter.

En matrett inneholder alltid **én eller flere** ingredienser. En ingrediens **kan inngå i flere** matretter, men trenger ikke å være med i noen. En ingrediens trenger ikke å være allergifremkallende (allergen-attributtet vil da være NULL).

Vi har altså et mange-til-mange forhold mellom **matrett**er og **ingrediens**er. Det er ikke lagret mer data om forholdet enn selve koblingene mellom matretter og ingredienser.

Vi tenker oss følgende tabeller. Det er tatt med eksempeldata for å illustrere mulig innhold.

matrett

id (PK) **Generert nøkkel**	navn	kategori	salgspris **DECIMAL / NUMERIC**
1	'Gulrotkake'	'søtbakst'	39.50
2	'Rundstykke m/ost'	'smørbrød'	25.00
3	'Rundstykke m/reker'	'smørbrød'	45.90
…	…	…	…

ingrediens

id (PK) **Generert nøkkel**	navn	allergen **Kan være NULL**
1	'hvetemel'	'gluten'
2	'hvitost'	'melk'
3	'reker'	'skalldyr'
4	'soyaolje'	'soya'
…	…	…

matrett_har_ingrediens

matrett_id (PK, FK) **Del av sammensatt PK**	ingrediens_id (PK, FK) **Del av sammensatt PK**	
1	1	*Gulrotkake inneholder hvetemel*
1	4	*Gulrotkake inneholder soyaolje*
2	1	*Rundstykke m/ost inneholder hvetemel*
2	2	*Rundstykke m/ost inneholder hvitost*

…	…

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (8% ~ 18 min) Skriv Java-klassene for entitetstypene som trengs for å representere eksempelet vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer. *Tips: I et rent mange-til-mange forhold trenger vi kanskje ikke en egen klasse for koblingstabellen!*

Vi ønsker kun å tilrettelegge for **enveis navigering** i Java mellom matrett og ingrediens, slik at en matrett inneholder de tilhørende ingredienser, mens en ingrediens **IKKE** inneholder de matretter den inngår i.

```
Løsning, poeng og merknader:

1   @Entity
    public class Matrett {

1       @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;

        private String navn;
        private String kategori;
2       private BigDecimal salgspris;

1       @ManyToMany(fetch = FetchType.EAGER)
1       @JoinTable(name = "examh24.matrett_har_ingrediens",
            joinColumns = @JoinColumn(name="matrett_id"),
            inverseJoinColumns = @JoinColumn(name="ingrediens_id"))
2       private List<Ingrediens> ingredienser;

        ...
    }

1   @Entity
    public class Ingrediens {

1       @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;

        private String navn;
        private String allergen;

        ...
    }
```

Du kan i de videre oppgavene anta at entitetsklassene inneholder de nødvendige konstruktører, gettere og settere, etc. du trenger i løsningene dine.

Vi antar at vi har en hjelpeklasse **KafeDAO**. Du skal lage et par metoder i denne. Du kan anta at en EntityManagerFactory kalt emf er opprettet og tilgjengelig.

b) (4% ~ 10 min) Skriv en metode **`hentAllergenerIMatrett(int matrettId)`** i MenyDAO som henter ut en liste av allergentypene som finnes i ingrediensene i en matrett. F.eks. skal matrett med id=1 (dvs. gulrotkake) gi svaret ['gluten', 'soya'].

Hjelp: Du kan anta at MenyDAO har en hjelpemetode `List<String> allergener(List<Ingrediens> ingredienser)` som tar en liste av ingredienser og returnerer en liste av allergener (der evt. duplikater og null-verdier er fjernet).

```
Løsning, poeng og merknader:

1   public List<String> hentAllergenerIMatrett(int matrettId) {

1       EntityManager em = emf.createEntityManager();

1       try {

2          Matrett matrett = em.find(Matrett.class, matrettId);

2          List<Ingrediens> ingredienser = matrett.getIngredienser();

1          List<String> allergener = allergener(ingredienser);
1          return allergener;

        } finally {
1           em.close();
        }
    }
```

c) (4% ~ 10 min) Skriv en metode **`hentMatretterIKategori(String kategori)`** i KafeDAO som henter ut en liste av matretter for en gitt kategori, f.eks. 'smørbrød'. Hvis ingen matrett i denne kategorien finnes, skal det returneres en **tom liste**.

```
Løsning, poeng og merknader:

1    public List<Matrett> hentMatretterIKategori(String kategori) {

1       EntityManager em = emf.createEntityManager();

        try {
3          String queryString
                     = "SELECT m FROM Matrett m WHERE m.kategori =
:kategori";

1          TypedQuery<Matrett> q = em.createQuery(queryString,
Matrett.class);

1          q.setParameter("kategori", kategori);

2          return q.getResultList();

        } finally {
1           em.close();
        }
    }
```

d) (4% ~ 10 min) Skriv en metode **`lagreIngrediens(String navn,String allergen)`** i KafeDAO som lagrer en ny ingrediens i databasen. Metoden returnerer det lagrede ingrediens-objektet. Hvis det allerede finnes en ingrediens i databasen med dette navnet, skal det ikke lagres noe, og null skal returneres.

Hjelp: Du kan anta at MenyDAO har en hjelpemetode `boolean finnesIngrediens(String navn)` som kan brukes til å sjekke om ingrediensen allerede finnes.

```
Løsning, poeng og merknader:

    public Ingrediens lagreIngrediens(String navn, String allergen) {

      EntityManager em = emf.createEntityManager();
1     EntityTransaction tx = em.getTransaction();

2     if (finnesIngrediens(navn)) {
           return null;
      }

      Ingrediens nyIngrediens;
      try {
1         tx.begin();

1         nyIngrediens = new Ingrediens(navn, allergen);
3         em.persist(nyIngrediens);

1         tx.commit();
      } finally {
          em.close();
      }
1     return nyIngrediens;
    }
```

Oppgave 4 – XML, JSON og NoSQL (25%, ~1 time)

4.1 XML og JSON – Flervalgsoppgaver (2% ~ 5 min)

a) Hvilken av følgende påstander er korrekt?

1. XML Schema benyttes til å beskrive lovlig oppbygning av JSON dokumenter
2. Følgende kan benyttes til å deklarere et XML dokument:
   <#xml version="1.0" encoding="UTF-8"#>
3. XML beskriver både struktur og presentasjon
4. Alle alternativer over er korrekte
5. Ingen av alternativene over er korrekte

```
2,5 Løsning, poeng og merknader:
5 – Ingen av alternativene over er korrekte
```

b) Hvilken av følgende påstander er korrekt for et velformulert XML dokument?

1. Kan ha flere rot elementer
2. Hvert element kan ha et antall attributter (men ikke med samme navn)
3. Elementer kan overlappe hverandre
4. Alle alternativer over er korrekte
5. Ingen av alternativene over er korrekte

```
2,5 Løsning, poeng og merknader:
2 – Hvert element kan ha et antall attributter (men ikke med samme navn).
Dette er riktig.
```

c) Hvilken av følgende påstander er **IKKE** korrekt for XML og navnerom?

1. Navnerom er innført for å unngå navnekollisjoner.
2. Et XML dokument kan ha to elementer med samme navn så lenge de er definert i forskjellige navnerom.
3. URL'er (URI) brukes for unik navngiving, og må være en nettadresse som eksisterer.
4. Følgende kan benyttes for å definere et «default» navnerom:
   xmlns = "http://www.hvl.no/dat107"
5. Følgende angir en mapping mellom et navnerom og et XML schema dokument:
   xmlns="http://www.hvl.no/dat107"
   xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
   xs:schemaLocation=http://www.hvl.no/dat107 menu.xsd

```
2,5 Løsning, poeng og merknader:
3 – URL'er (URI) brukes for unik navngiving, men de må IKKE være en
nettadresse som eksisterer
```

d) Hvilken av følgende er **IKKE** en innebygd datatype som kan benyttes i et XML Schema dokument:

   a. String
   b. Decimal
   c. Integer
   d. Primary Key
   e. Boolean

2,5 Løsning, poeng og merknader:
4 – «Primary Key» er **IKKE** en innebygd datatype som kan benyttes i et XML schema dokument.

4.2 XML og JSON – «Tenk og prøv selv» oppgaver

I «vedlegg 4.2» er det definert et XML Schema dokument (fil: ***menu.xsd***) som spesifiserer hvordan en applikasjon ønsker å representere og utveksle informasjon om menyer (f.eks. i en forbindelse med drift av restauranter).

Tips!

- *Default verdien for "minOccurs" og "maxOccurs" i en XML Schema definisjon er 1.*
- *xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="?"*

a) (3% ~ 7 min) Hva betyr det at et XML dokument er "velformet" og "gyldig"?

```
Løsning, poeng og merknader:
Syntaksregler for velformet XML
2 Hvert XML-dokument har ett og bare ett rot-element
2 Alle elementer har både start-tagg og slutt-tagg
2 Hvert element kan ha et antall attributter (ikke to attributter med
samme navn)
2 Kan ha nøstede elementer som i HTML, 2 men elementer kan ikke overlappe
```

b) (6% ~ 15 min) Lag et eksempel på et velformet og gyldig XML dokument som referer til (kobles) og følger XML Schema dokumentet "menu.xsd" (se vedlegg 4.2).
Eksemplet skal inneholde minst to forretter, to hovedretter og to desserter.

En av rettene skal innholde et allergen (f.eks. Gluten).

```
Løsning, poeng og merknader:
<?xml version="1.0" encoding="utf-8" ?>
<menu xmlns=http://eksamen.hvl.no/databaser
      xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
      xsi:schemaLocation="http://eksamen.hvl.no/databaser oppgave4-
1.xsd">
   <starters>
      <starter name="Garlic Bread">
         <description>Garlic bread with cheese</description>
         <price>100.00</price>
      </starter>
      <starter name="Shrimp coctail">
         <description>Shrimp cocktail with avocado</description>
         <price>150.00</price>
      </starter>
   </starters>
   <mainCourses>
      <mainCourse name="Beef steak">
         <description>Beef steak with pepper sauce</description>
         <price>250.00</price>
1         <allergens>
            <allergen>Gluten</allergen>
            <allergen>Laktose</allergen>
         </allergens>
      </mainCourse>
      <mainCourse name="Blab la bla">
         <description>Blaribla</description>
         <price>179.00</price>
      </mainCourse>
   </mainCourses>
   <desserts>
      <dessert name="Chocolate cake">
         <description>Chocolate cake with ice cream</description>
         <price>100.00</price>
      </dessert>
      <dessert name="Cheese cake">
         <description>Cheese cake duddli du</description>
         <price>110.00</price>
      </dessert>
   </desserts>
</menu>
```

```
1 – Deklarasjon (linje 1)
1 – Rot element <menu> (linje 1)
2 – Default namespace og mapping til XML schema (linje 2-4)
2 – Riktig bruk av start og slutt tager
2 – Riktig bruk av attributt «name»
1 – Riktig bruk av trestruktur
1 – Riktig bruk av hirarki mellom rett-type og rett, f.eks. <starters> og
<starter>, <mainCourses> og <mainCourse>
```

c) (3% ~ 7 min) Hvordan kan vi utvide XML Schema dokumentet i vedlegg 4.2 slik at

- Gyldig pris på de ulike rettene må være fra 100 til og med 500 kroner
- Gyldige allergener må hentes fra listen: “Gluten”, “Nuts”, “Lactose” and “Shellfish”.

```
Løsning, poeng og merknader:
Endre følgende linjer for “starterType”, “mainCourseType” og
“dessertType” fra …

<xs:element name="price" type="xs:decimal"/>

… til …
1 <xs:element name="price" type="xs:priceType"/>

Og legg til følgende:

4
<xs:simpleType name="priceType">
  <xs:restriction base="xs:desimal">
    <xs:minInclusive value="100.00"/>
    <xs:maxInclusive value="500.00"/>
  </xs:restriction>
</xs:simpleType>

Endre følgende linje fra …

<xs:element name="allergen" type="xs:string" …/>

… til …

1 <xs:element name="allergen" type="xs:allergenType" …/>

Og legg til følgende:
4
<xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="Gluten"/>
      <xs:enumeration value="Nuts"/>
      <xs:enumeration value="Lactose"/>
      <xs:enumeration value="Shellfish"/>
    </xs:restriction>
</xs:simpleType>
```

d) (3% ~ 7 min) Skriv ditt eksempel fra oppgave b) om til et JSON dokument. Bruk element- og attributtnavn som nøkkelnavn (der det er behov for dette).

Løsning, poeng og merknader:

```
{
  "starters": [
    {
      "name": "Garlic Bread",
      "description": "Garlic with cheese",
      "price": 100.00
    },
        {
      "name": "Shrimp coctail",
      "description": "Shrimp cocktail with avocado",
      "price": 150.00
    }],
  "mainCourses": [
    {
      "name": "Beef steak",
      "description": "Beef steak with pepper sauce",
      "price": 250.00,
      "allergens": ["Gluten", "Lactose"]
    },
         {
      "name": "Shrimp coctail",
      "description": "Shrimp cocktail with avocado",
      "price": 150.00
    }],
  "desserts": [
    {
      "name": "Chocolate cake",
      "description": "Chocolate cake with vanilla ice cream",
      "price": 100.00
    }]
}
```

1.5 – Riktig bruk av rot Objekt
1.5 – Riktig bruk av nøkkel/verdi par
1.5 – Riktig bruk av array [] for oppramsing av retter («starters», «mainCourses» og «desserts»
1.5 – Riktig bruk av array [] for «allergens»
1.5 – Riktig bruk av datatyper (String, descimal)
1.5 – Riktig bruk av trestruktur
1.0 – For å ikke gjenta entals navn på element, f.eks. «starter», «mainCourse» og «dessert» (tar bare ekstra plass, gir ingen verdi)

4.3 NoSQL – Flervalgsoppgaver (2% ~ 5 min)

a) Hvilken av følgende påstander er korrekt med henhold til NoSQL databaser?

1. Datasett kan distribueres over flere noder/ datamaskiner (sharding)
2. Samme data kan kopieres til flere noder (replikering)
3. NoSQL er en sekkebetegnelse på en rekke alternativer til relasjonsdatabaser
4. Det finnes 4 hovedtyper for NoSQL databaser: Nøkkel-verdi, Kolonne, Dokument og Graf/node databaser.
5. Alle alternativer over er korrekte

```
2,5 Løsning, poeng og merknader:
5 – Alle alternativer over er korrekte
```

b) Hvilken av følgende påstander er korrekt?

a. MongoDB er en graf/node basert NoSQL database
b. Amazon DynamoDB er en kolonne basert NoSQL database
c. Cassandra er en kolonne basert NoSQL database
d. Neo4J er en nøkkel-verdi basert NoSQL database
e. Alle alternativer over er korrekte

```
2,5 Løsning, poeng og merknader:
3 – Cassandra er en kolonne basert NoSQL database.

MongoDB er en dokument basert NoSQL database (ikke graf/node).
Amazon DynamoDB er en nøkkel-verdi basert NoSQL database (ikke kolonne
basert).
Neo4J er en graf/node basert NoSQL database, og ikke nøkkel-verdi.
```

c) Hvilken av følgende påstander er **IKKE** korrekt angående distribuerte databaser (kluster)?

1. I en ikke-delt arkitektur vil hver datamaskin/ node ha selvstending lagring av data. Denne arkitekturen benyttes som oftes av NoSQL databaser.
2. I en delt-disk arkitektur vil flere datamaskiner/ noder være koblet til samme data kilde. Denne arkitekturen kan blant annet benyttes for relasjonsdatabaser (f.eks. Oracle RAC).
3. CAP teoremet går ut på at man KUN kan velge 2 av de 3 følgende egenskapene når man designer et distribuert system:
   KONSISTENS (C - Consistency), TILGJENGELIGHET (A – Availability) og PRODUKTIVITET (P – Productivity)
4. Når vi tar utgangspunkt i CAP teoremet vil relasjonsdatabaser (stort sett) ha et CA design

```
2,5 Løsning, poeng og merknader:
3 - PRODUKTIVITET er IKKE en del av CAP teorem. P-en står for Partisjons
toleranse.
```

d) Hvilken av følgende påstander er korrekt om NoSQL databasen Cassandra?

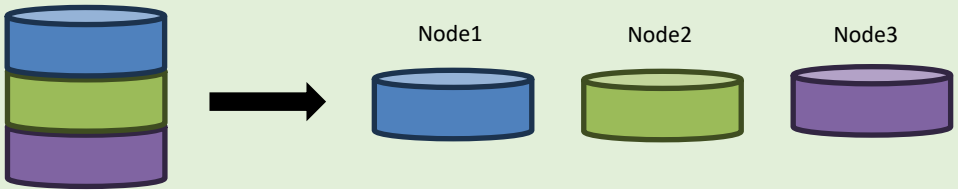
1. Cassandra er en kolonne basert database
2. Cassandra lagrer data fysisk i kolonner i stedet for rader
3. Cassandra ble lansert av Facebook I 2008, som et open-source prosjekt
4. Cassandra benytter spørrespråket CQL, som ligner på SQL
5. Alle alternativer over er korrekte

```
2,5 Løsning, poeng og merknader:
5 – Alle alternativer over er korrekte om Cassandra.
```

4.4 NoSQL - «Tenk selv» oppgaver

a) (2% ~ 4 min) Forklar kort hva som menes med "Sharding" når vi snakker om distribuerte database systemer?

```
Løsning, poeng og merknader:
Sharding er en type partisjonering (4) der man distribuerer et dataset ut
over flere noder (4). Dette kalles for horisontal skalering (2).
```



b) (2% ~ 4 min) Hva er fordelene med en distribuert database , fremfor en en-node løsning?

```
Løsning og poeng:
2 Last balansering:   Spre lasten over flere noder/ ressurser
2 Skalering:          Kunne serve et større antall brukere (samtidig)
2 Data redundans:     Sørge for at man har mer en en kopi av data
2 Tilgjengelighet:    Unngå «single-point of failure»
2 Vedlikehold:        Kan oppgradere en og en server (uten nedetid).
```

c) (2% ~ 4 min) Hva er ulempene med en distribuert database, sammenlignet med en en-node løsning?

```
Løsning og poeng:
Kompleksitet (5): Overhead og kompleksitet i å sette opp shards,
vedlikeholde dataene på hvert shard, og riktig ruting av forespørsler på
tvers av disse shards.
Konsistens(3): Opprettholdelse av datakonsistens kan være vanskelig,
spesielt i situasjoner med samtidige oppdateringer, noe som kan føre til
inkonsistente data.
Feilhåndtering(2): Feil i nettverket eller en av nodene kan påvirke hele
systemet, og det kan være utfordrende å håndtere slike feil effektivt.
Distribuert system må hondtere partisjons toleranse (P-en i CAP teorem).
NB! Her har man også fått poeng (2) for f.eks. «sikkerhet» og/eller
«ytelse» hvis man har tatt med dette (men med maks 10 poeng)
```

Vedlegg 4.2: (Filnavn: menu.xsd)

```
<?xml version="1.0" encoding="utf-8" ?>
<xs:schema targetNamespace="http://eksamen.hvl.no/dat107"
        xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="menu" type="menuType"/>

  <xs:complexType name="menuType">
    <xs:sequence>
      <xs:element name="starters" type="startersType"/>
      <xs:element name="mainCourses" type="mainCoursesType"/>
      <xs:element name="desserts" type="dessertsType"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="startersType">
    <xs:sequence>
      <xs:element name="starter" type="starterType" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="mainCoursesType">
    <xs:sequence>
      <xs:element name="mainCourse" type="mainCourseType" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="dessertsType">
    <xs:sequence>
      <xs:element name="dessert" type="dessertType" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="starterType">
    <xs:sequence>
      <xs:element name="description" type="xs:string"/>
      <xs:element name="price" type="xs:decimal"/>
      <xs:element name="allergens" type="allergensType" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="name" type="xs:string" use="required"/>
  </xs:complexType>

  <xs:complexType name="mainCourseType">
    <xs:sequence>
      <xs:element name="description" type="xs:string"/>
      <xs:element name="price" type="xs:decimal"/>
      <xs:element name="allergens" type="allergensType" minOccurs="0"/>
    </xs:sequence>
```

```
    <xs:attribute name="name" type="xs:string" use="required"/>
  </xs:complexType>

  <xs:complexType name="dessertType">
    <xs:sequence>
      <xs:element name="description" type="xs:string"/>
      <xs:element name="price" type="xs:decimal"/>
      <xs:element name="allergens" type="allergensType" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="name" type="xs:string" use="required"/>
  </xs:complexType>

  <xs:complexType name="allergensType">
    <xs:sequence>
      <xs:element name="allergen" type="xs:string" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```